

Leitura do acelerômetro MPU6050 com ESP32

Explicação do código, bloco por bloco

Instrumentação Eletrônica para Engenharia — Entrega 1

Visão geral

O projeto tem quatro arquivos e cada um resolve uma parte do problema. O firmware lê o sensor e imprime números na porta serial. Um script em Python pega esses números e salva em disco. Outro script transforma o arquivo salvo no gráfico que a entrega pede. O quarto arquivo só diz ao PlatformIO qual é a placa e qual biblioteca baixar.

Arquivo	Onde roda	O que faz
<code>platformio.ini</code>	computador	configura a compilação
<code>src/main.cpp</code>	ESP32	lê o sensor e imprime CSV
<code>capturar.py</code>	computador	grava o CSV em arquivo
<code>plot_dados.py</code>	computador	desenha o gráfico

O caminho do dado é sempre o mesmo: o MPU6050 mede, manda por I2C para a ESP32, a ESP32 converte e manda por USB para o computador, o Python grava e depois plota.

Ordem de uso:

1. `pio run -t upload` grava o firmware na placa.
2. `python capturar.py` faz o ensaio e salva `dados.csv`.
3. `python plot_dados.py` gera `grafico.png`.

1 Bloco 1 — platformio.ini

```
1 [env:esp32doit-devkit-v1]
2 platform = espressif32
3 board = esp32doit-devkit-v1
4 framework = arduino
5
6 monitor_speed = 115200
7
8 lib_deps =
9     electroniccats/MPU6050@^1.3.1
10
11 upload_port = COM3
12 monitor_port = COM3
```

Listing 1: platformio.ini

Esse arquivo não é código, é receita. O PlatformIO lê cada linha e decide o que baixar e como compilar.

platform escolhe a família do chip e traz junto o compilador **xtensa**, que gera código para o processador da ESP32. São centenas de megabytes, baixados uma vez só, na primeira compilação.

board é o modelo exato da placa. Define quanta flash existe, qual o cristal, e o mapa de pinos. Trocar essa linha muda o binário gerado.

framework = arduino é o que permite escrever **setup()**, **loop()**, **Serial.print()** e **delay()**. Sem ele o programa teria que ser escrito com as funções do ESP-IDF, que são bem mais verbosas.

lib_deps é a única dependência do projeto. O PlatformIO baixa a biblioteca MPU6050 da Electronic Cats direto do registro dele e a coloca em **.pio/libdeps/**. O **^1.3.1** aceita qualquer versão 1.x a partir da 1.3.1, mas nunca a 2.0, que poderia quebrar a interface.

As duas últimas linhas fixam a porta COM3. Sem elas o PlatformIO tenta adivinhar, e quando há mais de um dispositivo USB-serial ligado ele erra.

2 Bloco 2 — src/main.cpp (37 linhas)

Esse é o firmware inteiro. Ele faz três coisas: acorda o sensor, lê os três eixos, imprime. Nada mais.

2.1 Includes (linhas 1–3)

```
1 #include <Arduino.h>
2 #include <Wire.h>
3 #include <MPU6050.h>
```

`Arduino.h` traz `setup()`, `loop()`, `Serial`, `delay()` e `millis()`. Na IDE do Arduino esse include é automático; no PlatformIO, que compila `.cpp` de verdade, ele precisa estar escrito.

`Wire.h` é a biblioteca de I2C. O MPU6050 fala I2C, e é por aqui que a comunicação acontece.

`MPU6050.h` é a biblioteca do sensor. Ela já inclui internamente o `I2Cdev`, que é a camada que lê e escreve os registradores.

2.2 Pinos (linhas 5–6)

```
5 #define SDA_PIN 21
6 #define SCL_PIN 22
```

Na DevKit v1 os pinos 21 e 22 são os de I2C por padrão. SDA carrega os dados, SCL carrega o clock. Usar `#define` em vez de escrever 21 e 22 soltos no meio do código deixa claro o que cada número significa, e se a ligação mudar basta editar aqui.

2.3 A constante 16384 (linhas 8–9)

```
8 // na faixa de +-2g o sensor entrega 16384 contagens por g
9 #define LSB_POR_G 16384.0
```

Esse número não foi escolhido, ele sai da conta. O conversor analógico-digital do MPU6050 tem 16 bits com sinal, ou seja, o valor lido vai de -32768 a $+32767$. Na faixa de ± 2 g essa escala inteira representa 4 g de ponta a ponta. Logo:

$$\frac{32768}{2 \text{ g}} = 16384 \text{ contagens por g}$$

O ponto zero no final (`16384.0`) é o detalhe que mais derruba gente em C. Se estivesse escrito `16384`, o compilador trataria como inteiro, e `ax / LSB_POR_G` viraria divisão inteira: uma leitura de 16400 contagens daria 1 em vez de 1,001, e tudo abaixo de 16384 daria 0. Com o `.0` a constante é um `double`, o `int16_t` é promovido, e a divisão sai em ponto flutuante.

2.4 O objeto do sensor (linha 11)

```
11 MPU6050 mpu;
```

Cria o objeto que representa o sensor. O construtor sem argumento assume o endereço I2C `0x68`, que é o endereço quando o pino AD0 do módulo está em GND ou solto. Se o AD0 for ligado em 3V3 o endereço vira `0x69` e a linha teria que ser `MPU6050 mpu(0x69);`.

2.5 A função `setup()` (linhas 13–22)

```
13 void setup() {
14     Serial.begin(115200);
15     delay(2000);
16
17     Wire.begin(SDA_PIN, SCL_PIN);
18     mpu.initialize();
19     mpu.setFullScaleAccelRange(MPU6050_ACCEL_FS_2);
20
21     Serial.println("t_ms,ax_g,ay_g,az_g");
22 }
```

O `setup()` roda uma vez, logo depois que a placa liga ou reseta.

Linha 14. Abre a porta serial a 115200 bits por segundo. Esse número aparece em três lugares do projeto e os três têm que bater: aqui, no `monitor_speed` do `platformio.ini`, e no `ser.baudrate` do `capturar.py`. Se um deles divergir, o resultado é texto embaralhado na tela.

Linha 15. Dois segundos parados. Parece desperdício, mas o conversor USB-serial da placa leva um tempo para o Windows reconhecer a porta depois do reset. Sem essa espera, as primeiras linhas impressas se perdem, incluindo o cabeçalho do CSV.

Linha 17. Liga o barramento I2C com a ESP32 como mestre. A frequência padrão é 100 kHz, folgada para jumper de protoboard.

Linha 18. A linha mais importante do arquivo. Quando o MPU6050 é alimentado ele acorda em modo de suspensão: o bit `SLEEP` do registrador `PWR_MGMT_1` vem em 1 de fábrica, e nesse estado todas as leituras voltam zeradas. O `initialize()` zera esse bit, escolhe a fonte de clock e deixa o sensor medindo. Esquecer essa chamada é o erro mais comum de quem começa com esse sensor: o código compila, roda, não dá erro nenhum, e os três eixos ficam parados em zero.

Linha 19. Fixa a faixa em ± 2 g. É a faixa com melhor resolução ($61\mu\text{g}$ por contagem) e a certa para medir gravidade, que nunca passa de 1 g. Para medir impacto ou vibração forte seria preciso subir para ± 8 g ou ± 16 g, e aí o `LSB_POR_G` da linha 9 teria que mudar junto.

Linha 21. Imprime o cabeçalho do CSV. É a primeira linha do arquivo e serve para quem abrir no Excel saber o que é cada coluna.

2.6 A função `loop()` (linhas 24–37)

```
24 void loop() {
25     int16_t ax, ay, az;
26     mpu.getAcceleration(&ax, &ay, &az);
27
28     Serial.print(millis());
29     Serial.print(",");
30     Serial.print(ax / LSB_POR_G, 4);
31     Serial.print(",");
32     Serial.print(ay / LSB_POR_G, 4);
33     Serial.print(",");
34     Serial.println(az / LSB_POR_G, 4);
35
36     delay(50);
37 }
```

O `loop()` roda para sempre, repetindo enquanto a placa estiver ligada.

Linha 25. Três variáveis de 16 bits com sinal. O tipo foi escolhido para casar exatamente com o que o sensor devolve: nem maior, nem menor.

Linha 26. Faz a leitura. A função lê seis registradores de uma vez (de `0x3B` a `0x40`), junta os bytes alto e baixo de cada eixo e guarda o resultado. O `&` antes de cada variável passa o

endereço dela na memória, e é isso que permite a função escrever em três variáveis de uma vez só, coisa que um `return` normal não faria.

Linhas 28–34. Montam a linha do CSV. O `millis()` devolve quantos milissegundos se passaram desde que a placa ligou, e vira a coluna de tempo. Cada valor bruto é dividido por 16384 para virar g. O segundo argumento do `Serial.print`, o número 4, é a quantidade de casas decimais. A última chamada é `println` em vez de `print`, e é ela que fecha a linha com a quebra que o Python vai usar para separar as amostras.

Linha 36. Espera 50 ms antes de repetir, o que dá mais ou menos 20 amostras por segundo. Medindo os tempos no arquivo gerado, o intervalo real fica em torno de 51,6 ms, porque a leitura I2C e a impressão também consomem tempo e o `delay` só conta a partir do fim delas. A taxa efetiva é de aproximadamente 19,4 Hz. Para esse ensaio a diferença não importa, já que o tempo verdadeiro de cada amostra vai gravado na primeira coluna.

2.7 O que o firmware não faz

Vale registrar o que ficou de fora de propósito, porque a entrega pede só a leitura do acelerômetro:

- não lê o giroscópio, embora o chip tenha um;
- não faz calibração nem desconta offset;
- não aplica filtro de nenhum tipo, os valores são crus;
- não verifica se o sensor respondeu.

O último item tem consequência prática. Se o cabo estiver solto, o programa continua imprimindo, só que com todos os eixos em zero. O sintoma de falha é esse, e não uma mensagem de erro.

3 Bloco 3 — `capturar.py` (46 linhas)

Esse script existe por um motivo específico. O jeito óbvio de salvar os dados seria `pio device monitor > dados.csv`, mas o PlatformIO recusa redirecionamento de saída, e mesmo quando funcionava o PowerShell gravava o arquivo em UTF-16 com os avisos do PlatformIO misturados no meio. Ler a porta direto com a biblioteca `pyserial` resolve os dois problemas de uma vez.

3.1 Argumentos (linhas 8–9)

```
8 porta = sys.argv[1] if len(sys.argv) > 1 else "COM3"
9 saida = sys.argv[2] if len(sys.argv) > 2 else "dados.csv"
```

`sys.argv` é a lista do que foi digitado na linha de comando. O `sys.argv[0]` é o nome do script, o `[1]` é o primeiro argumento, e assim por diante. Rodando `python capturar.py` sem nada, o script usa `COM3` e `dados.csv`. Rodando `python capturar.py COM5 ensaio.csv`, ele usa o que foi passado.

3.2 Abrir a porta sem resetar errado (linhas 11–19)

```
11 # O DTR e o RTS dessa placa vao no GPIO0 e no EN. Se o pyserial ativar
12 # os dois na abertura, a ESP32 cai em modo de gravacao e nao envia nada.
13 ser = serial.Serial()
14 ser.port = porta
15 ser.baudrate = 115200
16 ser.timeout = 1
17 ser.dtr = False
18 ser.rts = False
19 ser.open()
```

Esse é o trecho menos óbvio do projeto inteiro, e vale entender direito.

A ESP32 precisa de dois sinais para ser gravada: o EN, que reseta o chip, e o GPIO0, que decide se o boot é normal ou se entra no gravador. Para não obrigar o usuário a apertar botão toda vez, as placas DevKit trazem um par de transistores que liga o DTR e o RTS do conversor USB-serial nesses dois pinos.

O problema é que o `pyserial`, quando abre a porta do jeito normal (`serial.Serial("COM3", 115200)`), ativa DTR e RTS juntos. Essa combinação é exatamente a que joga a placa em modo de gravação. Foi o que aconteceu no primeiro teste: a porta abria, a leitura rodava, e não vinha nenhum byte.

A solução está na ordem das linhas. Criar o objeto sem passar a porta (linha 13) não abre nada. Os parâmetros são preenchidos um a um, o DTR e o RTS são postos em `False`, e só depois o `open()` é chamado. A porta abre com os dois sinais já desligados.

3.3 O pulso de reset (linhas 21–25)

```
21 # pulso no EN para a placa reiniciar e a captura começar do inicio
22 ser.rts = True
23 time.sleep(0.1)
24 ser.rts = False
25 ser.reset_input_buffer()
```

Com o DTR desligado, o GPIO0 fica em nível alto, que é a condição de boot normal. Mexer só no RTS reseta a placa sem risco de cair no gravador.

O RTS vai para `True`, o que puxa o EN para baixo e segura o chip em reset. Cem milissegundos depois o RTS volta para `False`, o EN sobe, e a placa reinicia rodando o firmware. O `reset_input_buffer()` joga fora o que já tinha chegado, para a captura começar limpa.

O ganho disso é que cada execução do script começa com o `millis()` zerado na placa. Sem o reset, o tempo da primeira amostra seria um número qualquer, dependendo de há quanto tempo a placa estava ligada.

3.4 O laço de leitura (linhas 32–42)

```
32 try:
33     while True:
34         linha = ser.readline().decode(errors="replace").strip()
35         if linha.count(",") != 3:  # descarta as mensagens de boot da ESP32
36             continue
37         arquivo.write(linha + "\n")
38         arquivo.flush()
39         n += 1
40         print(n, linha, end="      \r")
41 except KeyboardInterrupt:
42     pass
```

Linha 34. O `readline()` espera até chegar uma quebra de linha, ou até estourar o `timeout` de 1 segundo. O que volta são bytes crus, e o `decode` transforma em texto. O `errors="replace"` evita que um byte corrompido derrube o programa no meio do ensaio. O `strip()` tira a quebra de linha do fim.

Linha 35. O filtro. Toda linha válida do firmware tem exatamente três vírgulas, porque tem quatro campos. As mensagens que o bootloader da ESP32 imprime ao ligar não seguem esse formato: a linha `rst:0x1 (POWERON_RESET),boot:0x13 (...)` tem uma vírgula, e a linha `clk_drv:0x00,q_drv:0x00,...` tem cinco. Contar vírgula é o teste mais barato que separa as duas coisas. O cabeçalho `t_ms,ax_g,ay_g,az_g` passa no teste, e é isso que se quer: ele fica gravado como primeira linha do arquivo.

Linhas 37–38. Escreve e força a gravação em disco na hora. Sem o `flush()`, o Python guardaria vários kilobytes em memória antes de escrever, e um Ctrl+C no momento errado perderia o fim do ensaio.

Linha 40. O argumento `end` termina com o caractere `\r`, que faz o cursor voltar para o começo da linha em vez de pular para a próxima. O resultado é um contador que se atualiza no mesmo lugar da tela, em vez de rolar centenas de linhas. Os espaços antes dele apagam o resto de uma linha anterior que tenha sido mais comprida.

Linhas 41–42. Ctrl+C no terminal levanta a exceção `KeyboardInterrupt`. Capturando ela aqui, o script sai do laço em vez de morrer com um traceback feio, e ainda dá tempo de fechar o arquivo e a porta nas linhas seguintes.

4 Bloco 4 — plot_dados.py (51 linhas)

4.1 Leitura do arquivo (linhas 14–29)

```
14 for linha in open(arquivo):
15     campos = linha.strip().split(",")
16     if len(campos) != 4:
17         continue
18     try:
19         ms = int(campos[0])
20         gx = float(campos[1])
21         gy = float(campos[2])
22         gz = float(campos[3])
23     except ValueError:
24         continue # pula a linha do cabeçalho
25
26     t.append(ms / 1000.0)
27     x.append(gx)
28     y.append(gy)
29     z.append(gz)
```

Percorrer o arquivo com `for linha in open(...)` lê uma linha por vez, sem carregar tudo na memória.

O `split(",")` quebra a linha nas vírgulas e devolve uma lista de pedaços. A verificação de tamanho descarta qualquer linha que não tenha exatamente quatro campos.

O `try` resolve o cabeçalho sem precisar de uma regra especial para ele. Quando o Python tenta fazer `int("t_ms")`, ele levanta `ValueError`, o `except` captura, e a linha é pulada. Uma linha truncada por um cabo desconectado no meio da captura cai no mesmo lugar.

A ordem importa. Os quatro valores são convertidos primeiro para variáveis soltas, e só depois de os quatro darem certo é que entram nas listas. Se a conversão fosse feita direto dentro do `append`, uma linha que falhasse no terceiro campo já teria deixado o tempo e o X gravados, e as quatro listas ficariam com tamanhos diferentes. O gráfico sairia deslocado sem avisar.

4.2 Tempo relativo (linhas 34–35)

```
34 inicio = t[0]
35 t = [v - inicio for v in t]
```

A primeira amostra chega com uns 2 segundos no relógio, por causa do `delay(2000)` do firmware. Subtrair o valor inicial de todos faz o gráfico começar em zero, que é como se esperar um ensaio.

4.3 O gráfico (linhas 37–51)

```
41 plt.axhline(1.0, color="gray", linestyle="--", linewidth=0.8)
42 plt.axhline(0.0, color="gray", linestyle="--", linewidth=0.8)
43 plt.axhline(-1.0, color="gray", linestyle="--", linewidth=0.8)
```

As três linhas tracejadas em -1 , 0 e $+1g$ são o gabarito visual do ensaio. Com o sensor parado, cada curva tem que pousar em cima de uma delas. Qualquer desvio visível já é erro de medida, e fica fácil de apontar no relatório.

O `savefig` vem antes do `show`, e essa ordem não é acidental: em vários backends do matplotlib a janela interativa esvazia a figura ao ser fechada, e um `savefig` depois disso gravaria uma imagem em branco.

5 Bloco 5 — O ensaio

O enunciado pede a leitura com o sensor em posições coincidentes com o vetor da gravidade. São seis posições, uma para cada sentido dos três eixos. Com o sensor parado, o eixo que estiver na vertical marca $\pm 1\text{ g}$ e os outros dois ficam perto de zero.

Posição do módulo	X (g)	Y (g)	Z (g)
deitado, componentes para cima	0	0	+1
deitado, de cabeça para baixo	0	0	-1
de pé sobre a borda, X para baixo	+1	0	0
borda oposta, X para cima	-1	0	0
de pé sobre a borda, Y para baixo	0	+1	0
borda oposta, Y para cima	0	-1	0

O procedimento é rodar o `capturar.py`, segurar cada posição por uns cinco segundos, passar devagar de uma para a outra, e encerrar com Ctrl+C. Os trechos parados viram patamares no gráfico e os movimentos viram os transitórios entre eles.

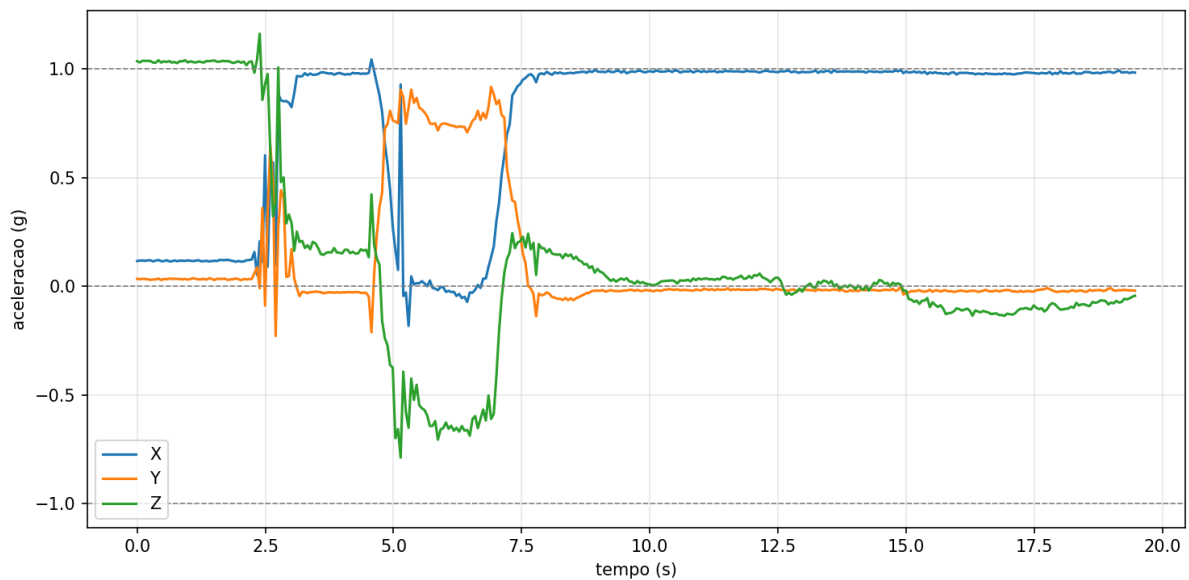


Figura 1: Captura de exemplo. Os patamares em +1 g de Z no início e de X no fim correspondem a duas das seis posições da tabela; a agitação no meio é a transição feita à mão.

6 Limitações conhecidas

Essas são as coisas que o código não resolve, e que valem ser citadas no relatório como fontes de erro.

O chip não é um MPU6050. O registrador `WHO_AM_I` respondeu `0x70`, que corresponde a um MPU6500. É um clone muito comum nos módulos GY-521 vendidos hoje. Os registradores do acelerômetro são os mesmos, por isso o código funciona sem alteração, mas as características de ruído e as faixas do filtro interno são diferentes das do datasheet do MPU6050. A tabela do item 2 deveria citar o datasheet do MPU6500.

Não há calibração. Com a protoboard parada sobre a mesa, o eixo X marcou em torno de $+0,12\text{ g}$ em vez de zero. Parte disso é inclinação real da mesa e parte é o erro de offset de fábrica, que o datasheet especifica em até $\pm 80\text{ mg}$. Separar as duas contribuições exigiria nivelar o sensor com bolha e repetir a medida em posições opostas.

A amostragem não é rigorosamente periódica. O `delay(50)` garante um intervalo mínimo, não um intervalo exato. O período medido ficou em $51,6\text{ ms}$. Para uma análise no domínio da frequência isso seria um problema, e a solução correta seria disparar a leitura por interrupção de timer.

A faixa de $\pm 2\text{ g}$ satura. Numa captura em que o módulo foi sacudido de propósito, os picos ficaram ceifados em exatamente $2,00\text{ g}$. O valor não é o real, é o teto da escala. Para medir impacto seria preciso trocar a faixa na linha 19 do firmware e ajustar o `LSB_POR_G` da linha 9.

O script assume COM3. A porta está escrita direto no código como valor padrão. Em outro computador, ou até nessa máquina com outro dispositivo USB ligado antes, o número muda. O jeito de contornar é passar a porta como argumento.